Tidal: A Deadline-Contracted Batch Tier for Unmodified LLM Serving Engines

Gurunath Lunkupali Venugopal
gurunathrajagopal@gmail.com

***Abstract*—A serving node sized for interactive traffic runs far below its throughput ceiling most of the time: our online-only GPU baseline emits a third of its node's offline ceiling, and scheduler budget left unspent in an iteration is gone. Three capacities order the node: its throughput saturated with offline work, co-serving batch behind live traffic, and serving live traffic alone. A batch tier can lift the node from the third toward the first without forking the engine. Commercial providers sell that gap as a second product, a batch API at half price with a 24-hour completion window; self-hosted deployments have none, and the systems that harvest idle capacity (HyGen, ConServe) fork the engine and promise no completion time. Tidal is that contract over unmodified vLLM: an OpenAI-wire `/v1/batches` front end, admission that *refuses* windows the system's own observed rate cannot meet, and per-job least-laxity escalation. On A6000 nodes serving Qwen2.5-7B, a gateway sidecar recovers 69.3% of the node's steady-state offline ceiling at $1.18\times$/$1.23\times$ online p50/p99 (flat 20 req/s, cross-node); under a diurnal trace at 12.1 req/s mean offered load it recovers 78.1%. We also implement the identical policy *inside* the engine as a scheduler plugin and compare both placements. Code and evidence are open source.**

***Index Terms*—LLM serving, batch inference, deadline scheduling, GPU utilization, vLLM**

I. Introduction

An engine serving interactive chat traffic runs its scheduler every few tens of milliseconds; in each iteration it may decode one token for each of a few dozen live requests while its token budget — the number of tokens the hardware could process in that iteration at acceptable latency — sits in the thousands. That slack is the margin commercial providers monetize as a second product: submit a file of requests, receive results within 24 hours, pay half the online price. Three gaps keep a self-hosted vLLM deployment from selling it:

- **The budget is perishable and mostly unspent.** Unused tokens in an iteration are gone. Our in-engine instrumentation shows online-only traffic on our testbed leaving most of $\tau = 2048$ tokens idle in a typical iteration (Section V-F); vLLM ships no surface that sells the remainder — its offline runner requires dedicating the machine, and an SLA-tier proposal for its scheduler was closed without adoption [25].
- **Best-effort co-location is not a product.** Harvesting systems — HyGen [8] and ConServe [9] recover 78–88% of dedicated-offline throughput behind online traffic — expose no per-job deadline, no admission decision, and no promise a customer can plan around; both are forks of the engine. Deadline-aware schedulers (Niyama [11]) target second-to-minute horizons and degrade rather than refuse.
- **The naive path is catastrophic.** vLLM's own priority field, used alone, costs $27.3\times$/$22.6\times$ online p50/p99 on our testbed (Section V-D): admission priority does not bound resident-request contention. Something must meter how much batch work exists at all.

Code, receipts, and raw evidence: `github.com/rajagurunath/tidal`

The missing piece is not a better preemption mechanism — ConServe's layer-wise safepoints already yield in single-digit milliseconds — but a *contract*: a specific job, a specific deadline, admission that refuses infeasible work, and scheduling that spends exactly as much online headroom as that deadline requires. The contract is what makes the tier sellable. A customer submits an overnight batch because "done by morning" is dependable enough to plan around; our motivating client, lazycode [27, 28] — a coding agent that plans work with a realtime model and executes it on provider batch APIs at the discount — exists only because that window is dependable. And the contract changes what the scheduler may do: where disaggregation separates prefill from decode in *space*, a deadline-contracted tier may separate work in *time* (Section VI).

Tidal implements the contract with deliberately classical machinery. Per batch job it tracks *laxity* — time remaining minus work remaining at the observed completion rate [21]. At admission, a job whose projected completion exceeds its window is refused with an error naming the arithmetic. In scheduling, urgency ramps only inside a short escalation horizon of remaining *slack*: an on-track job never rises above the lowest priority band no matter how long it has run, an at-risk job escalates early, continuously, and independently of other jobs, and a token-bucket floor guarantees forward progress at maximum urgency — progress that feeds back into laxity and stabilizes the job at the floor rather than oscillating.

We implement this policy at two placements and compare them — the axis no prior co-serving system evaluates, since each evaluates only its own. *Technique A* is a sidecar gateway over a stock engine: batch items enter as low-priority requests, throttled at 1 Hz by AIMD over public metrics. *Technique B* moves admission into the engine through vLLM's pluggable scheduler interface: a subclass wraps the stock scheduler and caps batch tokens per iteration against a latency model fitted continuously from the engine's own step timings. Neither placement forks the engine.

Contributions:

1) **A customer-facing batch contract for self-hosted serving** (Section III-B): OpenAI-wire `/v1/batches` with durable per-job state, laxity-based admission refusal, horizon-scaled escalation, and a progress floor — semantics neither the co-serving nor the deadline-scheduling literature exposes. The contract is validated in mechanism but not in outcome: no load regime we constructed discriminates escalation end-to-end (Section V-E).
2) **The measured price of the no-fork placement at GPU scale** (Section V-C): 69.3% of a same-node steady-state offline ceiling at $1.18\times$/$1.23\times$ online p50/p99 under flat load, 78.1% under a diurnal trace at 12.1 req/s mean offered load — inside the 78–88% band the fork-based literature reports, over an engine we never modified.
3) **Placement as an empirical design axis** (Section V-D): the same policy outside versus inside the engine on identical traces is a throughput/latency *frontier*, not a ranking — at capacity on the CPU testbed the gateway holds online p99 to $1.97\times$ where the in-engine scheduler reaches $9.9\times$ while harvesting 26 more points of ceiling; the one GPU point shows the frontier moves with hardware (the $2.54\times$ CPU median tax is $1.18\times$ on the A6000).
4) **An audited measurement discipline for co-serving harvest** (Section V-A): normalize to a *steady-state-tail* offline denominator, and treat any harvest above 100% of a claimed ceiling as a failed ceiling, not a system win. This rule changed our own reported result.

II. Motivation

A. One budget, two products

A vLLM V1 iteration schedules up to `max_num_batched_tokens` (τ) tokens across running and waiting requests; τ is sized so an iteration's latency fits the inter-token target. A running decode consumes one token per iteration, a prefill up to a chunk; the KV cache separately bounds how many requests may be resident. The token budget is the perishable good.

Two distinct mechanisms make the slack harvestable. Filling residual budget with batch *prefill* chunks spends idle *compute*: prefill is compute-bound, and chunked-prefill schedulers already interleave it under a latency budget [1]. Co-batching batch *decodes* alongside online decodes spends idle *memory bandwidth*: with B concurrent decodes, one HBM traversal of the weight matrices serves B tokens, so a thicker decode batch moves the projection and MLP multiplies from thin, GEMV-like shapes toward higher-arithmetic-intensity GEMM regimes — throughput rises with B at nearly flat iteration latency until compute, or the KV-cache reads that do *not* amortize (each request reads its own history), bind instead. Neither effect is monotonic or free; both are bounded by the online latency budget. Our GPU result — 1,408 batch output tok/s added at $1.18\times$ online p50 — is *consistent with* this arithmetic; we have not run the controlled batch-size sweep that would demonstrate it mechanistically.

a) A roofline reading.: The roofline model bounds attainable performance by the lower of a compute ceiling and a bandwidth ceiling selected by a kernel's arithmetic intensity [2]. The GEMV→GEMM effect above is exactly a move rightward along that intensity axis, and co-batching is the only lever an unmodified engine offers for making it. Roofline is therefore the frame, GEMV→GEMM the mechanism, and the following ordering the evidence:

$$\begin{aligned}\text{hardware roofline} &\geq \text{engine offline ceiling}\\ &\geq \text{co-serving} \geq \text{online-only.}\end{aligned}$$

We do not measure the leftmost term. That would require a validated compute-and-bandwidth model of the deployed A6000 at the engine's precision, which we did not build, and we make no claim about it beyond the inequality. The second term we do measure: the same engine, model, and node saturated with offline work and nothing else sustains 2049.6, 2031.5, and 2057.0 output tok/s in steady state across our three nodes (Section V-A). It is an operational stand-in — a measured lower bound sitting under the hardware roofline, and the only denominator we are entitled to divide by. The distance below it is what this paper is about. Online-only traffic at 20 req/s Poisson leaves the node emitting 687.6 output tok/s — its own completions summed over the baseline run's 6-minute window — 33.5% of that node's 2049.6 tok/s offline ceiling, at a mean of 15.64 resident requests; the CPU rig at chat-scale load is starker, a mean of 0.41 (Figure 1). The gateway lifts harvested work to 1,408 batch output tok/s under that same flat load, and to 1,607 under the diurnal trace at 12.1 req/s mean offered load (Section V-C). The chain is an ordering of regimes, not an additive budget: each term is measured on its own request mix, so the online and batch token rates are not summable, and the online rate is set by offered load rather than by capacity — which is why co-serving's price shows up in latency and not in online throughput.

B. Why best-effort is not a product

Best-effort co-location delivers whatever slack happens to exist; the customer learns at hour 24 how much that was. The obvious repair — "priority rises every hour, parity at hour 24" — fails twice: it escalates small on-track jobs pointlessly and escalates large at-risk jobs too late, with full service beginning exactly when time has run out. Feasibility is a relation between remaining work and remaining capacity — which is laxity, and which is why the same arithmetic serves as the admission test. We require of a contract: (R1) urgency is a function of remaining work versus remaining capacity, never of the clock alone; (R2) infeasible work is refused at submission, not accepted and missed; (R3) escalation is per-job and continuous, so at-risk jobs neither wait for a synchronized threshold nor flood the online tier together.

C. The placement question

Engine schedulers act per iteration ($\sim$10–100 ms) with exact visibility of the token budget and KV state; an external

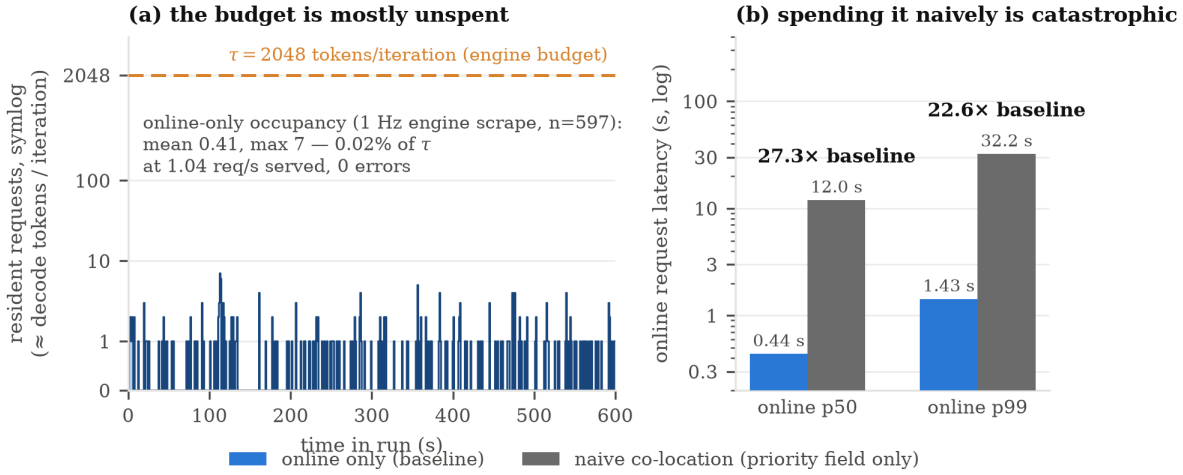

Fig. 1. **The budget is mostly unspent — and spending it naively is catastrophic.** Left: resident online requests (≈ decode tokens per iteration) under online-only traffic on our testbed — mean 0.41, i.e. 0.02% of the $\tau = 2048$ budget. Right: the online latency price of naive co-location (vLLM's priority field alone), 27.3×/22.6× p50/p99 (Section V-D) — admission priority does not bound resident-request contention, which is why something must meter how much batch work exists at all.

controller acts at 1 Hz on aggregate metrics. Intuition says in-engine must dominate. But the engine's own priority preemption already covers token-granularity interference between controller ticks, and an external controller survives engine upgrades untouched. No prior co-location work isolates this variable; Section V-D measures it, and the intuition does not survive contact.

III. Design

A. Overview

Figure 2 shows the datapath. Batch jobs enter through an OpenAI-wire `/v1/files`+`/v1/batches` API (any OpenAI SDK works with a changed `base_url`), are validated line-by-line, checked for feasibility, and stored durably (SQLite WAL; the store is a repository interface, so Postgres is a DSN change). Items progress `pending` → `inflight` → `succeeded`/`failed` with idempotent terminal transitions. Online traffic never traverses the gateway. Placement A stops at the engine's public API; placement B adds an in-engine scheduler plugin. Both placements run the same contract arithmetic.

B. The contract: laxity at admission and in scheduling

For each active batch b with deadline D_b, remaining items R_b, and observed completion rate r_b (sliding window; with no completion history the drain term is dropped — jobs escalate on evidence or deadline proximity, never on a missing denominator):

$$\text{laxity}(b) = (D_b - \text{now}) - R_b/r_b \tag{1}$$

$$\text{urgency}(b) = \text{clamp}\left(1 - \frac{\text{laxity}(b)}{H},\, 0,\, 1\right) \tag{2}$$

$$\text{priority}(b) = 100 \to 1 \text{ linearly with urgency} \tag{3}$$

with H = escalation horizon ($6\,\text{h} \ll 24\,\text{h}$ window) and (online = 0; `sla_strict` allows 0 at urgency 1).

a) Admission (R2).: At `POST /v1/batches`, projected completion R/r_{global} is checked against the window with a configurable margin; infeasible jobs are rejected with an error naming the arithmetic. No history → accept (the system cannot judge).

b) Scheduling (R1, R3).: An on-track job (laxity $\geq H$) never rises above priority 100 — it exerts no *escalation* pressure whatever the clock says. The horizon, not the window, scales urgency: scaling by the window is wall-clock aging in disguise. Escalated jobs climb independently; above urgency 0.9 a token bucket guarantees a minimum admission rate, and the floor's assured progress feeds back into laxity, stabilizing the job at the floor. Escalation applies at submission time (vLLM fixes a request's priority at admission); the backlog, where laxity lives, is by construction the unsubmitted portion. Figure 3 contrasts the resulting curves with wall-clock aging. Co-location has a latency price even at the lowest band — Section V measures it; the contract's property is never spending *more* online headroom than the deadline requires, not zero cost.

C. Technique A: external control

A 1 Hz loop scrapes three public metrics (waiting count, KV usage, completion rate), smooths them, and adjusts a target in-flight batch count by AIMD: halve on detected online queueing or a high KV watermark, increment below a low watermark. The waiting metric is not priority-partitioned, so the halving conservatively assumes every queued request is ours. Items are claimed in deadline order, grouped by shared prompt prefix to recover some prefix-cache locality through arrival order (unablated against HyGen's engine-side ordering [8]), and submitted at their batch's current laxity priority. Engine unavailability opens a circuit; in-flight items re-queue idempotently. Cost: one metrics scrape and one $O(\text{jobs})$ laxity pass per second — negligible beside a single decode step.

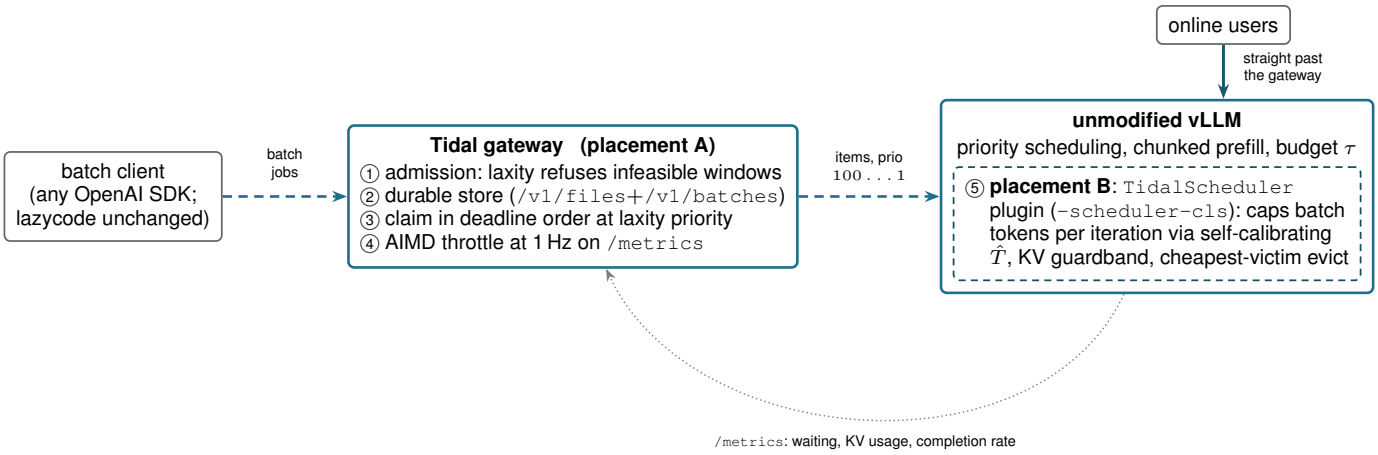

Fig. 2. **One policy, two placements.** Batch jobs enter through an OpenAI-wire gateway that ① refuses infeasible windows at admission, ② stores jobs durably, ③ claims items in deadline order at their laxity priority, and ④ throttles dispatch by AIMD over the engine's public metrics. Placement A stops there: the engine is stock vLLM, and online traffic never traverses the gateway. Placement B moves admission inside the engine ⑤ as a scheduler plugin that caps batch tokens per iteration against a self-calibrated latency model. Both are plugins; neither forks the engine.

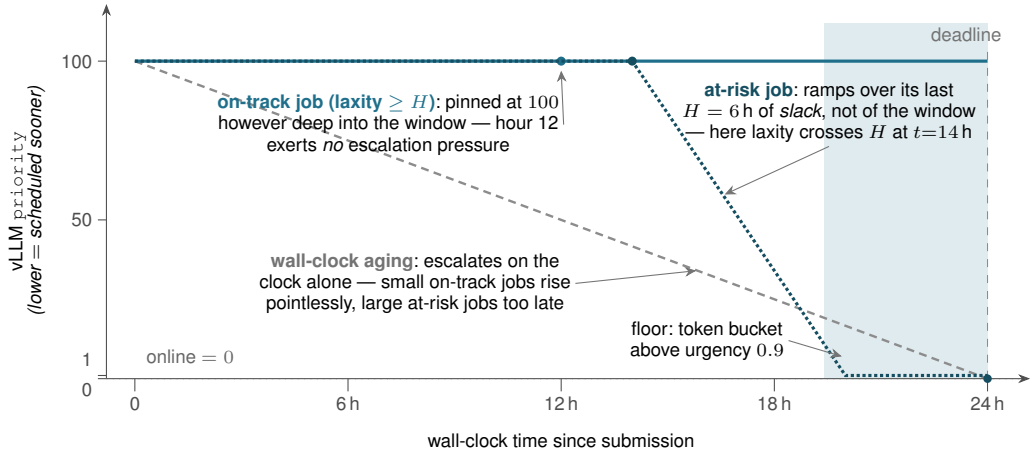

Fig. 3. **The contract in one picture.** Wall-clock aging (gray dashes) escalates every job on the clock alone. Laxity escalation separates the cases: an on-track job (solid) stays at the lowest band for the entire window, while an at-risk job (dotted) ramps linearly over its last $H = 6$ h of *slack* — beginning when its laxity crosses H, not at any fixed hour — to one notch below online. Above urgency 0.9 (shaded) a token bucket guarantees a minimum admission rate, whose assured progress feeds back into laxity and stabilizes the job at the floor; `sla_strict` allows the dashed final step to priority 0 — parity — exactly at the deadline.

D. Technique B: in-engine work conservation

`TidalScheduler` subclasses the V1 scheduler behind `-scheduler-cls` and wraps — never reimplements — the stock `schedule()` (Algorithm 1). Requests with priority ≥ 1 are the batch class. Each iteration solves for the largest batch admission x with $\hat{T}(P_{\text{on}} + x, C) \leq \text{TBT}_{\text{budget}}$, where $\hat{T}(P, C) = k_1 P + k_2 P(P{+}C) + k_4(P{+}C) + k_5$ is ConServe's context-aware form [9] — it prices the attention cost a pure token count misses — but fitted *continuously* from the engine's own $(P, C, \text{step-time})$ stream: ring buffer, periodic least squares, non-negativity clamps, and an R^2 gate with a conservative static fallback, removing the offline profiling sweep HyGen and ConServe require. With zero online requests resident the cap lifts entirely (offline mode: fill τ). A KV guardband (EWMA of online demand, clamped 5–30%) stops batch admission before online bursts can starve, and proactive eviction picks the batch victim with the least non-prefix-cached work. Enforcement is at chunk granularity — we bound which requests the stock scheduler sees, not its inner token loop — which is what keeps the wrapper robust to upstream drift (it survived an upstream jump of roughly 1,700 commits during development; porting to release v0.26.0 required one keyword argument) at the interference cost Section V-D quantifies. Per-iteration overhead: one closed-form cap solve and two list partitions; the least-squares refit is periodic and off the step path.

E. Robustness

The failure paths are part of the design, consolidated here. *Model misfit*: if $\hat{T}$'s fit fails its R^2 gate, the cap falls back

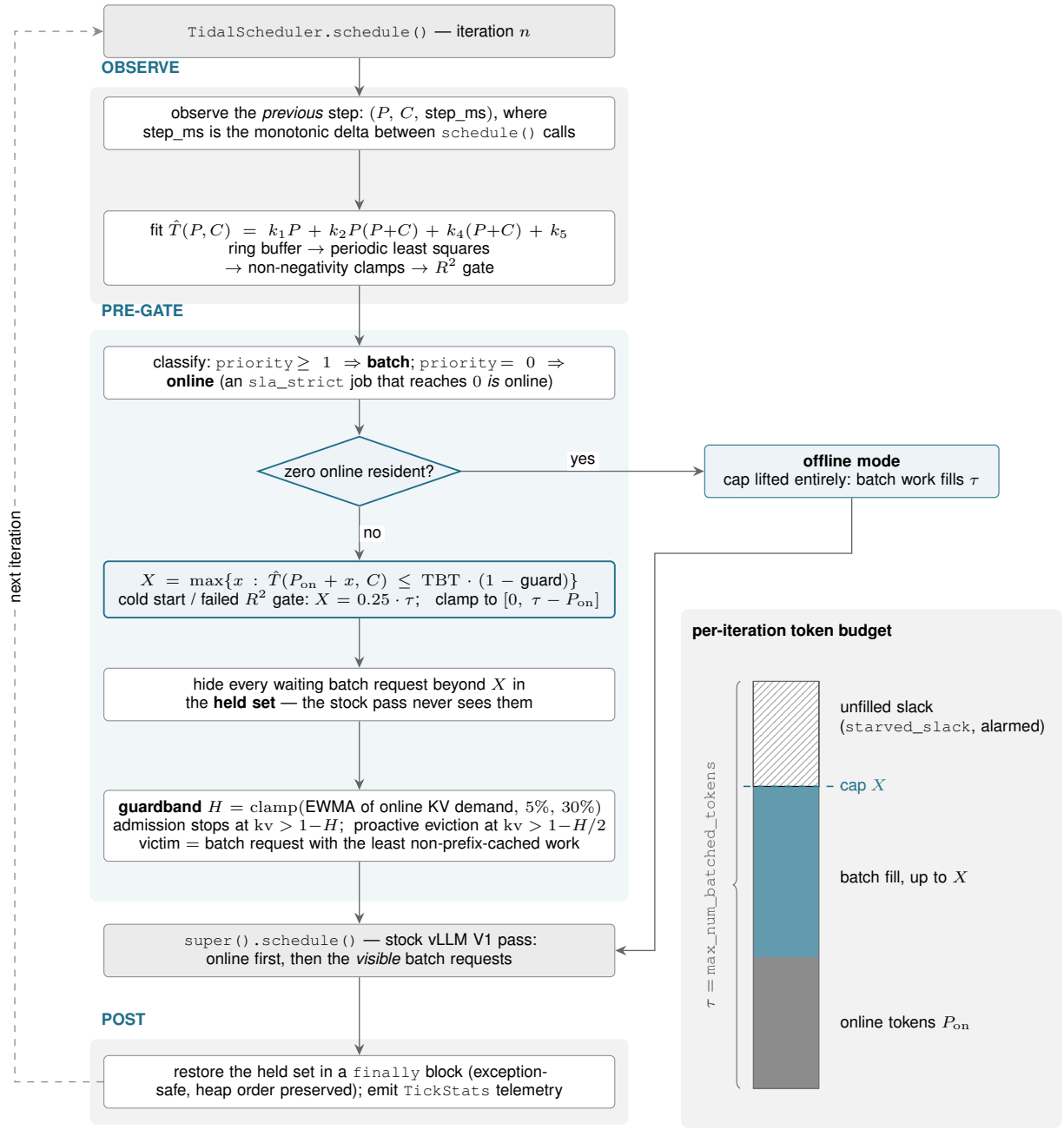

Fig. 4. **Technique B: one engine iteration.** `TidalScheduler` wraps rather than reimplements the stock `schedule()`. Before the stock pass it feeds the previous step's $(P, C, \text{step\_ms})$ to a continuously refitted latency model, solves that model for the largest batch-token cap X that keeps the step inside the guarded inter-token budget, and hides every waiting batch request beyond X — lifting the cap entirely when no online request is resident. A KV guardband stops batch admission before online bursts can be starved of blocks and evicts the cheapest batch victim under further growth. After the stock pass the held set is restored in a `finally` block. The side panel shows what X governs: online tokens occupy the bottom of the τ-token budget, batch work fills up to the cap, and whatever is left unfilled is reported as starved slack.

Algorithm 1 TidalScheduler pre-gate (one iteration)

$x \leftarrow \max\{x : \hat{T}(P_{\text{on}}{+}x,\, C) \le \text{TBT}_{\text{budget}}\}$
▷ $x \leftarrow \infty$ if no online resident
hide batch waiters beyond x; hide all batch if KV $> 1{-}H_{\text{kv}}$
out ← stock `schedule()`
▷ unmodified upstream pass
restore hidden waiters (exception-safe)
if KV $> 1 - H_{\text{kv}}/2$: evict cheapest batch victim
record $(P, C,$ step time); periodically refit $\hat{T}$, gate on R^2

to a conservative static value. *Cold start*: with no completion history the laxity drain term is dropped — escalate on evidence, never on a missing denominator. *Engine loss*: the circuit opens, in-flight items re-queue idempotently, and a restarted dispatcher re-claims resultless items (re-execution rides the prefix cache). *Duplicate control*: a second dispatcher against one store is data-safe but duplicates work; there is no ownership lease (roadmap). *Late results* for cancelled or expired items are recorded-then-discarded without corrupting counts. *Non-stable interfaces*: `scheduler_cls` is not a stable vLLM interface; technique B pins a version and probes signatures. What we deliberately did not build: ConServe's layer-wise safepoint preemption and incremental KV checkpointing address unbounded iteration times; chunked prefill bounds ours to one τ-sized step, and Section V-D argues sub-iteration control is exactly what technique B lacks.

IV. Implementation

Tidal is Python over unmodified upstream vLLM (v0.26 series): 424 tests including integration tests against a live engine, with the API layer wire-verified against the official OpenAI SDK. Item claims are single atomic updates; terminal transitions are idempotent, so duplicate or late results never drift `request_counts`; result-file attachment is first-writer-wins, so concurrent finalization cannot split a batch's outputs. Metering prices batch tokens at a configurable discount against an online price table; we make no claim about what discount a marginal-cost model would justify. The strongest wire-compatibility evidence is external: lazycode's Tidal adapter mirrors its Anthropic batch adapter nearly line-for-line — an existing batch-API client adopted the self-hosted tier as just another provider [28]. One obligation a fork never meets: vLLM attaches log handlers only to its own logger tree and runs the scheduler in a forked process, so an engine plugin injects logging configuration or runs blind. Operational semantics in full: Appendix B.

V. Evaluation

Five questions, in order of weight: **Q1** what does the no-fork gateway cost and recover at GPU scale (Section V-C)? **Q2** does harvest follow the online tide (Section V-C)? **Q3** how does placement change the price of the same policy (Section V-D)? **Q4** does the contract machinery change deadline outcomes (Section V-E)? **Q5** does the in-engine model calibrate itself (Section V-F)?

A. Testbeds, and defining the ceiling

Table I fixes both rigs. Requests are short chat completions; we measure total non-streaming request latency (on these configurations a request is a short prefill plus a short decode; a TTFT/TPOT split is not measured here).

a) Defining the ceiling.: Every "% of ceiling" below divides by the same node's *dedicated-offline* rate — and how that denominator is measured decides the result. A burst probe's whole-burst average is ramp-dominated: our probe drains 600 items at concurrency 512 with an 18.5 s makespan, so roughly half the window is pipeline fill, and the whole-burst figure (1438, 1427, and 1468 output tok/s across the three nodes) understates the node by 43%. The correct denominator is the probe's **steady-state tail** (last 10 s): 2049.6, 2031.5, and 2057.0 tok/s, a $\pm 0.62\%$ cross-node band — a tighter instrument as well as a correct one. We learned this by being wrong: an independent numerical audit of our first draft rejected the whole-burst denominator, and the tell was in our own table — a diurnal run "harvesting" 109% of it. The rule this yields is free to apply and the field does not apply it: **any co-serving harvest above 100% of a claimed ceiling is proof the denominator is not a ceiling.** The prior literature does not report how its ceilings were measured (Section VII). Both denominators and the audit trail are archived in the repository. Throughput is normalized per node; **latency ratios are cross-node** (no within-node baseline/co-serving pair exists in the evidence) and are labeled as such throughout.

B. Comparison points

Table II makes the positioning checkable rather than asserted. Our evaluation arms: `online_only` (baseline), `naive` (vLLM priority field alone — the policy-free control), technique A, technique B, and `offline_only` (the ceiling probe).

C. Q1–Q2: the no-fork price at GPU scale

The gateway recovers **69.3% of the node's steady-state offline ceiling** while holding the online tax to $1.18\times$/$1.23\times$ p50/p99 (Table III, Figure 5) — inside the 78–88% band HyGen and ConServe report for forked engines, from a sidecar over a stock container image. Under the diurnal trace, harvest rises to 78.1% — but the diurnal run's mean offered load is 12.1 req/s against the flat run's 20 (completion lengths are nearly identical, 56.0 vs 54.8 mean output tokens), so the two rows are different regimes, not a dose-response, and we do not present 78.1% as evidence that diurnal load helps per se.

a) The tide (Q2).: Per-minute batch completions anti-correlate with online load at $r = -0.85$ with the first-minute pipeline-fill bin excluded (minute 0 completes 771 items against 1667–1894 in every later minute; including it drags the correlation to -0.38, a startup artifact). The GPU tide is in *phase* with the CPU tide (-0.85 vs -0.95; Figure 6) but much shallower: batch completions split -6.4% between above- and below-mean-load minutes on GPU against -22% on CPU — consistent with the GPU having more absolute

TABLE I

TESTBEDS. THE CPU RIG IS FOR CONTROLLED MECHANISM AND PLACEMENT COMPARISONS ONLY; THE GPU RIG CARRIES THE CAPACITY AND LATENCY CLAIMS.

	CPU (mechanism/placement)	GPU (capacity/latency)
hardware	Apple M4 Pro, CPU-only build	RTX A6000 48 GB × 3 nodes (io.net)
model	Qwen2.5-0.5B-Instruct	Qwen2.5-7B-Instruct
engine	vLLM v0.26.1 @ `e6d67fdd`	`vllm/vllm-openai:v0.26.0` image
budget, policy	τ=2048, priority, eager	τ=2048, priority
online load	Poisson, seeded, 1–3 req/s	Poisson, seeded, 20 req/s
windows	10 min	6 min flat; 20 min diurnal; warmup 45 s
metric	total request latency	total request latency (non-streaming)

TABLE II

WHAT EACH SYSTEM EXPOSES. *AS REPORTED BY THOSE PAPERS, FOR FORKED ENGINES, DENOMINATOR METHODOLOGY UNREPORTED; †STEADY-STATE-TAIL DENOMINATOR (SECTION V-A), UNMODIFIED ENGINE, DIURNAL ROW AT LOWER MEAN OFFERED LOAD (12.1 VS 20 REQ/S). BLENDSERVE IS OFFLINE-ONLY; ITS NUMBER IS % OF A PROFILED PRACTICAL OPTIMUM, AND ITS ONLY DEADLINE NOTION IS A POOL-WIDE BLENDING WINDOW.

	per-job deadline	admission refusal	online tier	engine change	placements evaluated / harvest
HyGen	—	—	✓	fork	1 / 78–88%*
ConServe	—	—	✓	fork (∼9.2k LOC)	1 / 78–88%*
BlendServe	window only	—	—	modified backend	1 / 86.6–90.8% of T_o
Niyama	per-request (s–min)	degrades	QoS classes	fork	1 / —
Tidal	**24 h, per job**	**refuses**	✓	**none (plugin/sidecar)**	**2** / 69.3%; 78.1%†

TABLE III

GPU RESULTS (A6000, QWEN2.5-7B, ONLINE 20 REQ/S POISSON). "% OF CEILING" USES EACH NODE'S OWN STEADY-STATE-TAIL PROBE (SECTION V-A). LATENCY RATIOS ARE CROSS-NODE. THE DIURNAL ROW IS A 20-MINUTE RUN, PEAK 20 REQ/S, MEAN OFFERED LOAD 12.1 REQ/S (n=14,498, ZERO ERRORS) — NOT DIRECTLY COMPARABLE TO THE FLAT ROW.

condition	online p50	online p99	vs baseline	batch otok/s	% of ceiling
online_only (baseline)	0.753 s	1.570 s	1.00×	—	—
technique_a (flat)	0.886 s	1.927 s	1.18×/1.23×	1408	**69.3%**
technique_a (diurnal)	0.85 s	1.90 s	∼1.13×/∼1.21×	1607	**78.1%**

headroom at the diurnal run's 78.1% ceiling point (12.1 req/s mean offered load) than the CPU box had at 45%.

b) What the GPU campaign did not produce.: Technique B's compatibility canary passed on release v0.26.0 (one keyword-argument fix; 14 engine tests pass against both release and dev vLLM), but its matrix measurement — and the `offline_only` and `naive` GPU arms — were lost to a two-stage harness defect in the direct-submission path (Appendix A). The placement comparison therefore remains CPU-only, and the CPU naive arm stands as the policy-free control.

D. Q3: placement, and an inversion

Naive co-location is catastrophic despite correct priority ordering (Table IV, Figure 7): admission priority does not bound resident-request contention. Between the two placements the naive expectation — in-engine must dominate — is wrong on this hardware. Technique B's cap and guardband address channels that never bind here: peak KV usage stayed below 0.5%, so eviction never fired, and a resident batch request decodes every iteration regardless of the admission cap. The binding channel is *resident-decode contention*, and only sub-iteration participation control — precisely ConServe's layer-wise mechanism, which our no-fork constraint excludes — could bound it. Technique A holds latency in both regimes (1.76× vs 8.25× p99 at equal work; 1.97× vs 9.9× at capacity) because its control point — how many batch requests exist at all — is upstream of residency. In the capacity regime, technique B converts that residency into 26 more points of ceiling (71.2% vs 45.1%). Placement is a frontier (Figure 8): the gateway buys latency protection with throughput, the in-engine scheduler the reverse. The comparison is of two placements of *one policy*, not of two systems — and the GPU crossover remains unmeasured because the direct-submission GPU arms failed (Section V-C). The one GPU point shows the frontier moves: the median tax that costs 2.54× on CPU costs 1.18× on the A6000.

E. Q4: the contract, honestly

We attempted to demonstrate end-to-end deadline discrimination — a feasible-but-tight job that best-effort co-location misses and laxity escalation meets — and did not find a load

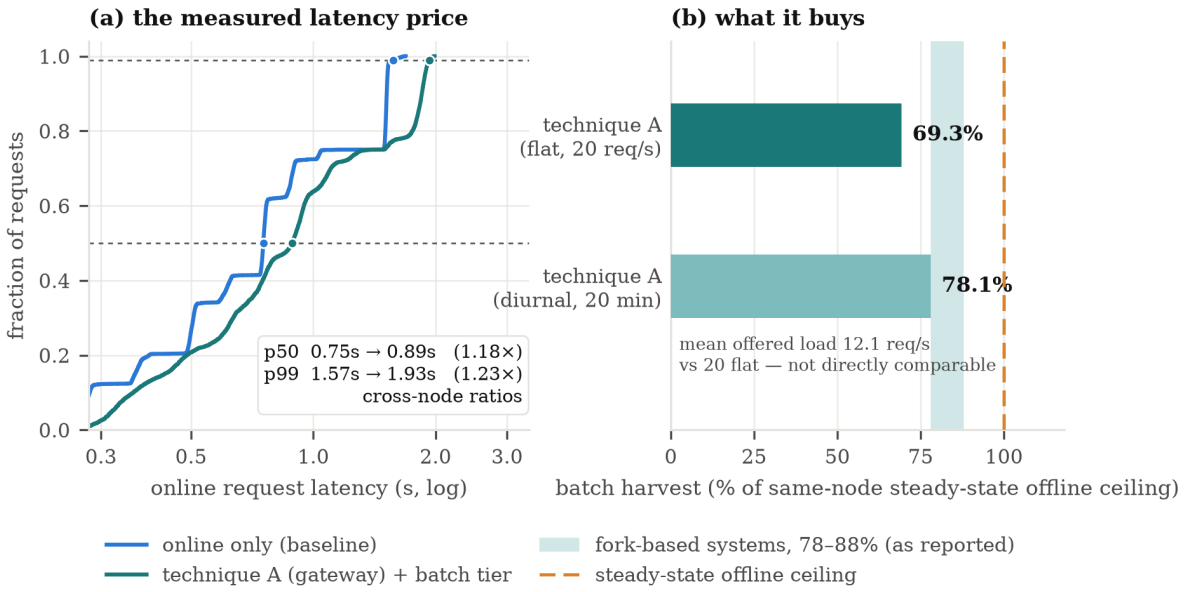

Fig. 5. **The headline trade at GPU scale.** Left: online request-latency CDF, baseline versus technique A (cross-node). Right: batch harvest as a fraction of the node's steady-state offline ceiling, with the band the fork-based literature reports shaded for reference.

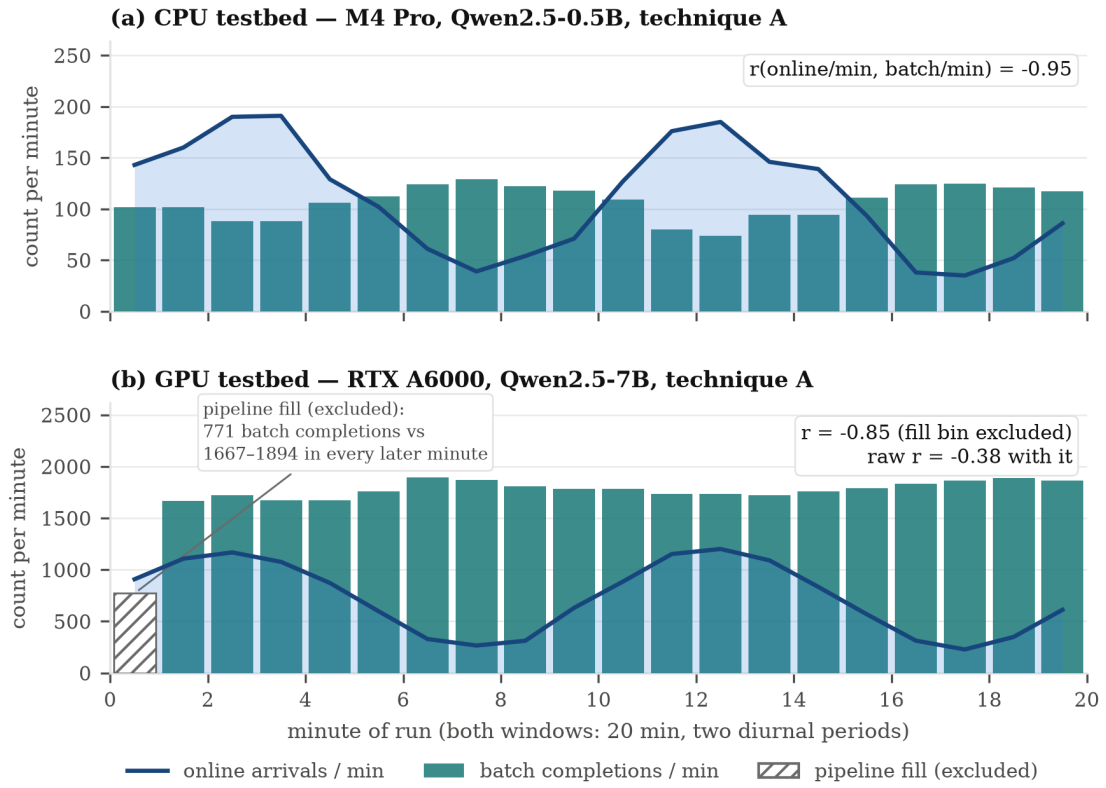

Fig. 6. **Batch fills the trough on both testbeds.** Per-minute batch completions (bars) against online arrivals (line): CPU $r = -0.95$ (top), GPU $r = -0.85$ with the first-minute pipeline-fill bin shaded and excluded (bottom). On the CPU run the AIMD target never moved from its cap — the tide is produced entirely by engine-level priority scheduling, the two-level division of labor observed directly.

regime on this testbed that discriminates. At 3 req/s with a 900-item/15-minute job, the batch completed with 3.5 minutes to spare under the heaviest online load we could apply; at 2–2.6 req/s every configuration completed early for *both* arms (Table V; full grid in Appendix D). **We therefore do not claim demonstrated deadline improvement.** What the runs do establish: the escalation machinery operates (per-tick priority traces show laxity bands engaging, minimum priority 66), the

TABLE IV

CPU PLACEMENT COMPARISON, ONE TABLE, TWO REGIMES: *equal-work* (ALL CONDITIONS COMPLETE THE IDENTICAL 1,200-ITEM BATCH; 10-MIN WINDOWS, 1 REQ/S) AND *capacity* (NON-DRAINING 3,000-ITEM POOL). CEILING: 4.43 ITEMS/S (68.5 OUTPUT TOK/S), DEDICATED-OFFLINE.

condition	regime	online p50	online p99	vs baseline (p50/p99)	% of ceiling
online_only	equal-work	0.44 s	1.43 s	1.00×/1.00×	—
naive	equal-work	11.97 s	32.24 s	27.3×/22.6×	—
technique A	equal-work	1.12 s	2.52 s	2.54×/1.76×	—
technique B	equal-work	1.26 s	11.80 s	2.86×/8.25×	—
technique A	capacity	—	—	p99 1.97×	45.1%
technique B	capacity	—	—	p99 9.9×	71.2%

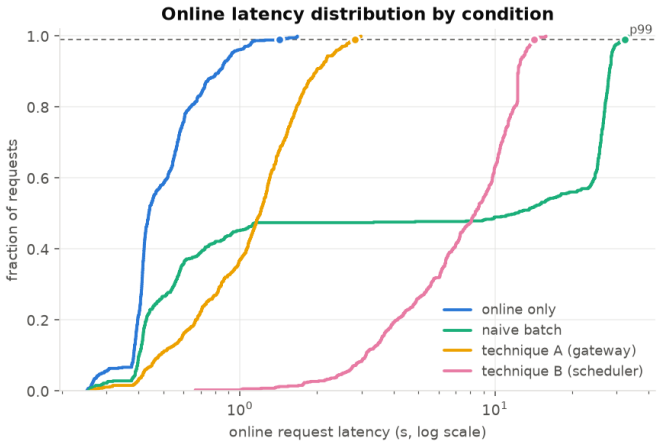

Fig. 7. Online request-latency distribution under each CPU condition. The medians degrade more than the tails everywhere — co-serving adds a near-constant per-step cost that dominates fast requests — so p99-only reporting, common in this literature, flatters every technique, including ours.

TABLE V

DEADLINE-STRESS RUNS (TECHNIQUE A, 15-MINUTE WINDOWS). EVERY FEASIBLE CONFIGURATION COMPLETED INSIDE THE WINDOW FOR BOTH ARMS; ESCALATION ENGAGED BUT WAS NEVER THE DIFFERENCE-MAKER. FULL FIVE-ROW GRID: APPENDIX D.

run	online load	items	completed	makespan	min. priority
saturated, best-effort	3.0 req/s	900	900/900	692 s	100
tight, best-effort	2.6 req/s	500	500/500	351 s	100
tight, laxity (+floor)	2.6 req/s	500	500/500	328 s	66

floor engages at its configured urgency, and admission refuses jobs the observed rate cannot serve. The A6000 runs validate slack harvesting under flat and diurnal load but still do not construct the discriminating regime; doing so — bursty load with genuinely scarce slack — remains the contract's open experiment.

F. Q5: self-calibration

From cold start, $\hat{T}$ passed its R^2 gate within the first capped phase of the integration run and settled at MAPE ≈ 0.11 on live traffic; the batch-token cap moved from the cold-start 40 to a converged 1 — correctly tight, since at a 60 ms budget with $\sim$25 ms online-only steps this box has little slack to sell, and the model discovered that (Figure 9). The same trace shows the offline-mode transition: when online traffic stops, the held set empties and batch fills the whole budget. Regime-change adaptation (model swap, thermal drift) is untested.

VI. DISCUSSION: WIDENING THE CONTROL PLANE

Everything measured above places the batch tier against one aggregated engine. The same contract widens along three axes, for which we have local evidence only (details, tables, and figures in Appendix C). **Fleets:** with two phase-shifted replicas, placement that follows slack harvested $+32\%$ over pinning all batch to one replica, with per-replica batch completions anti-correlating with that replica's own load ($r = -0.83, -0.56$) — mechanism proven on a CPU rig whose shared memory bus is a named confound on the latency side. **Disaggregation:** the three signals technique A scrapes have direct counterparts in NVIDIA Dynamo [20] (KVBM block metrics, router queue depth, per-worker throughput); the one non-rename nuance is that batch work is prefill-heavy, so the tier should target *prefill-worker* troughs. **Phase splitting:** a deadline legalizes an arbitrary gap between a request's prefill and its decode. A freeze/thaw spike — prefill in one process, KV persisted to disk, process killed, a fresh process thaws — produced bit-identical greedy decode with TTFT $31.8\times$ faster than recomputing ($7.277 \rightarrow 0.229$ s, median of 5). Total FLOPs are unchanged; only their placement in time moves, and the bill moves to storage (12,288 B/token at 0.5B; $\sim$2 orders worse at 70B-class). Disaggregation separates prefill from decode in space; a deadline-contracted batch tier separates them in time.

VII. RELATED WORK

Iteration-level scheduling. Orca introduced iteration-level batching [3]; Sarathi-Serve's chunked prefill and token budget are the structure vLLM V1 inherits [1]; FastServe brought preemptive MLFQ to inference [4]; InferCept's preemption taxonomy [23] underlies HyGen's KV handling. DistServe and Splitwise disaggregate phases in space [6, 7]; Llumnix rebalances by live migration [5]; LMCache and Mooncake [18, 19] own the KV-persistence mechanics our freeze/thaw spike deliberately reuses rather than rebuilds.

Online/offline co-serving. ConServe [9] is the closest system and the strongest published harvest–interference frontier: an offline-profiled context-aware latency model, layer-wise safepoint preemption, and incremental KV checkpointing, at

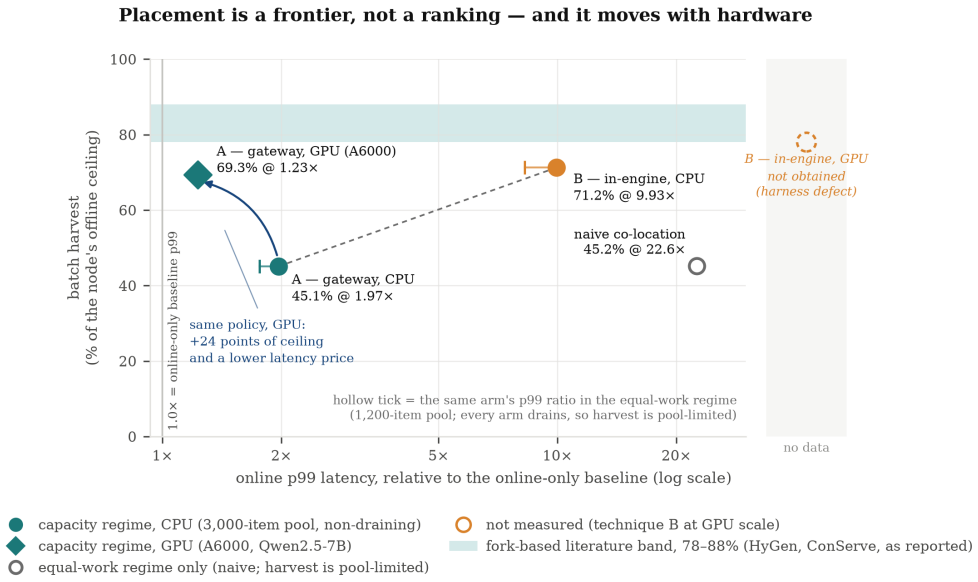

Fig. 8. **Placement is a frontier, not a ranking.** Online p99 ratio (log) against harvest, all measured arms, with the band the fork-based literature reports shaded. The same policy moves from the CPU point to the GPU point when the binding resource changes; technique B's GPU point was not obtained (open marker).

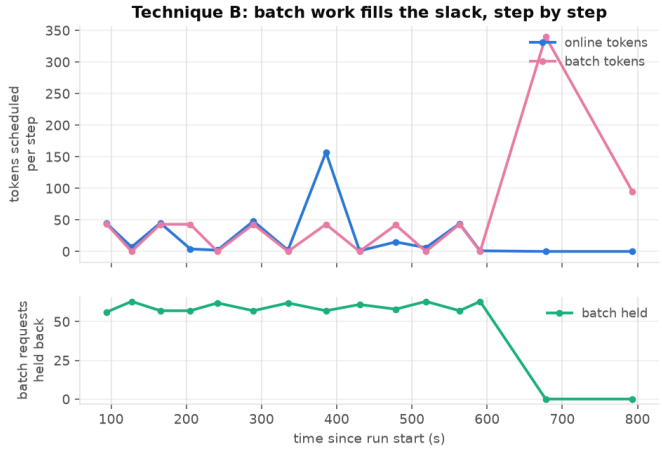

Fig. 9. Technique B, step by step: online and batch tokens scheduled per iteration (top) and batch requests held back by the pre-gate (bottom). Online traffic stops near $t = 600$ s; the held set empties and batch takes the whole budget — offline mode.

78–88% of ceiling within 25% of online-only latency. We take its numbers as the target our in-engine placement should reach rather than a claim we contest, and we differ on exactly three axes, none of which is a better preemption mechanism. First, ConServe's offline pool has no per-job identity: its only SLO is the online one, its offline metric is aggregate tokens per second, its `s` knob is an operator dial — nothing is ever refused. Tidal's contribution sits there (per-job windows, admission refusal, horizon-scaled escalation, a floor with laxity feedback). Second, ConServe is a ∼9.2k-LOC fork including in-model instrumentation; both our placements are unforked, and we treat that as a measured cost/benefit — our wrapper survived an upstream jump of roughly 1,700 commits at the price of chunk-granular enforcement, which is one side of our placement frontier. Third, ConServe evaluates one placement; we compare two. HyGen [8] shares ConServe's shape (profiled budget, prefix-aware offline order, starvation tempering) and the same three differences. OOCO [16] and Echo [17] are contemporaneous co-scheduling work, also contract-free.

Offline batch optimization. BlendServe [10] exploits the same physics from the opposite side: with *no online tier*, it reorders an offline pool so compute-heavy and memory-heavy requests overlap, reaching 86.6–90.8% of a profiled optimum. Tidal's setting inverts the degrees of freedom: the memory-bound side of our blend is pinned by an exogenous arrival process we cannot reorder, our control variable is an admission rate under a live latency constraint rather than a one-time request order, and where BlendServe's one deadline notion is a pool-wide blending window, our unit of promise is the individual job. The systems compose: BlendServe's compute-density order is the right *intra-job* claim order for our dispatcher, and its density metric suggests selecting batch items whose resource shape complements the current online mix — neither system implements that today.

Deadline-aware serving. QLM [13] (ILP queue reordering), Ascendra [14], SCORPIO [15], and closest, Niyama [11]: per-request deadlines across QoS classes at second-to-minute horizons, degrading under overload. None exposes day-scale per-job windows or admission refusal. FREESH [12] runs true LLF inside an engine for energy-aware scheduling of one request class — evidence the math runs in this setting, aimed at a different problem. Within vLLM itself: priority scheduling was motivated by batch/interactive co-location [24], an SLA-tier RFC was declined [25], and an online batch-API endpoint

remains an open PR [26] — demand without an assembled answer. Every individual mechanism here is published; the contribution is the contract, the placement measurement, and the audited synthesis.

VIII. Limitations

Six, one line each; full provenance in Appendix A. (1) Technique B's admission gating does not bound resident-request inflation — the dominant residual interference channel on the CPU testbed; bounding it requires sub-iteration control. (2) The contract is mechanism-validated, not outcome-demonstrated (Section V-E), and `sla_strict`'s latency price is unmeasured. (3) The placement inversion is CPU-only; technique B's GPU arm was lost to a harness defect. (4) The GPU ceiling is a steady-state tail of a probe burst, not a 6-minute dedicated run, and GPU latency ratios are cross-node. (5) The fleet experiment's latency side is confounded by shared memory bandwidth (thread-count isolation only). (6) Single engine, single model, one dispatcher per store (no lease); submission-time escalation cannot re-age already-queued items; `scheduler_cls` is not a stable interface.

IX. Conclusion

The batch API is the rare product where the entire margin is scheduling. Tidal implements its contract — admission, escalation, floor — over an unmodified open-source engine with classical real-time machinery, prices the no-fork placement at GPU scale, and is explicit about what remains undemonstrated. We offer it as usable infrastructure, a baseline for deadline-contracted serving, and a placement study whose GPU evidence currently covers the gateway but not the in-engine scheduler. What is not measured here is equally specific: the in-engine GPU matrix, blocked by our harness defect, and a load regime that discriminates escalation end-to-end.

Appendix A
Artifact and provenance

a) Harness defects.: (1) The direct-submission path fails at GPU scale twice-over: an unbounded drain (fixed with bounded salvage, 12 tests) and a remaining stall-after-warmup mode; it cost the `offline_only`, `naive`, and technique-B GPU arms. (2) macOS fleet isolation is thread-count only. (3) Diurnal-B and B-matrix GPU numbers are absent, blocked by (1). (4) Metrics evidence is our own 1 Hz instrumentation plus archived CaaS API responses and engine logs (no production observability stack on the dev cluster). (5) Two analysis defects found by an independent audit, not by us: a ramp-dominated probe average used as a ceiling and a pipeline-fill bin left inside a correlation — both corrected in Section V-A and Section V-C, both with tells we did not act on (a >100% harvest; a first bin completing 771 items against 1667–1894).

Appendix B
Operational semantics and the client

A restarted dispatcher re-queues resultless in-flight items; re-execution rides vLLM's prefix cache. Late results for cancelled or expired items are recorded-then-discarded without corrupting counts, and un-wedge finalization. One dispatcher per store is assumed; a second is safe for data, not for duplicated work (no ownership lease; roadmap). Figure 10 traces one batch through the gateway path end to end; Figure 11 closes the loop from client intent to applied code. The client's architecture — the planning algebra, rewrite-rule optimizer, wave scheduler, and batch-vs-cached-realtime economics — is the companion paper's subject [28].

Appendix C
The A-ladder: fleets, disaggregation, phase splitting

Table VI lays out the ladder; Figure 12 draws it. The ladder is the complement of technique B, not a competitor: B is maximum visibility over minimum scope (one engine's iteration); the ladder trades coarser signals for widening scope, ending in a decision B cannot express — parking work in time.

TABLE VI
The A-ladder: widening the scope over which the deadline-contracted tier places work, with the evidence status of each rung.

rung	scope	status
A0	single aggregated engine	measured on GPU (Section V-C)
A-F	fleet placement across replicas	mechanism proven locally (Table VII)
A-D	disaggregated-aware injection	design + Dynamo metric mapping (Table VIII)
A-P	phase-split freeze/thaw	spike-proven locally (31.8× TTFT)

a) A-F: fleet placement.: Two CPU replicas (Mac M4 Pro; thread-count isolation only — macOS cannot pin cores, so shared memory bandwidth is a named confound), phase-shifted diurnal load (period 600 s, phases $0/\pi$), 20-minute window, identical 4000-item pool. **This is not the GPU setup** (Qwen2.5-0.5B, peak 1 req/s, 16-token completions); its only job is the placement mechanism. Fleet placement (sticky prefix affinity + least-loaded + headroom filter, per-replica AIMD with replica-local circuits; 95 of the 424 tests) harvested 2494 in-window items versus pinned 1884 (+32%), with batch volume split nearly evenly (1239/1255) but *timed* into each replica's trough — per-replica correlation $-0.83/-0.56$ (Figure 13). The cost is real: blended online p50 3.41 s vs 1.44 s, p99 12.1 vs 3.7 — attributed to the shared-substrate confound (spreading batch loads both replicas' shared memory bus). GPU fleets with real isolation are where the latency side resolves; the mechanism claim is what this rig proves.

b) A-P: freeze/thaw.: vLLM CPU build, `Example Connector` (the renamed `SharedStorageConnector`), `kv_role=kv_both`, prefix caching disabled. Prefill in process 1 → KV persisted (23.6 MB for a 1973-token prompt: whole 128-token blocks only, 1920 cached tokens at exactly 12,288 B/token) → process killed → fresh process thaws → bit-identical greedy decode, TTFT $7.277 \rightarrow 0.229\,\text{s} = 31.8\times$

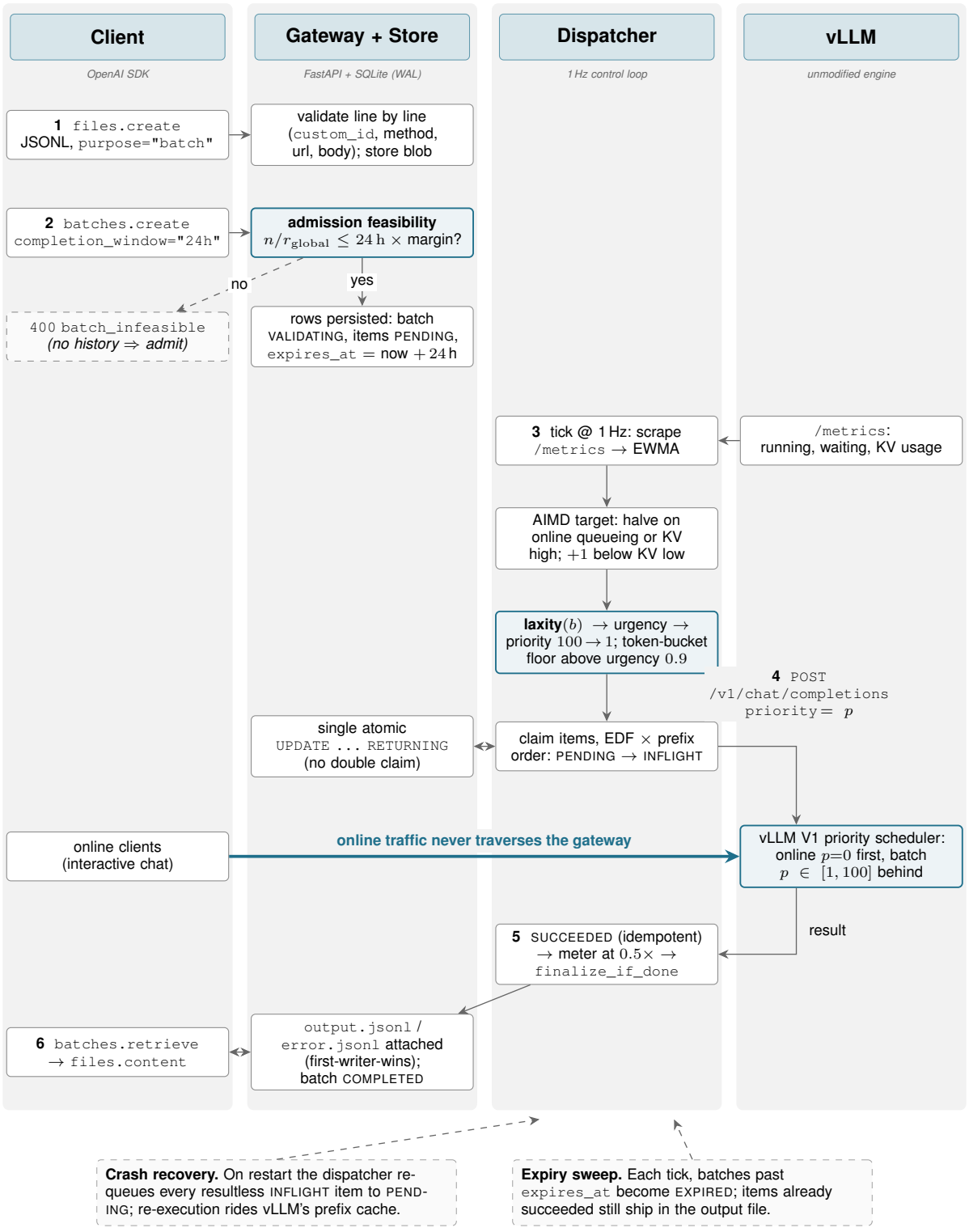

Fig. 10. **Technique A: the batch request lifecycle, end to end.** A client uploads a JSONL file and creates a batch; the gateway validates every line and applies the laxity admission test, refusing with `batch_infeasible` any job whose projected completion exceeds the window times a margin, and otherwise persisting PENDING items with a 24-hour `expires_at`. A 1 Hz dispatcher scrapes the engine's public metrics, sets an AIMD in-flight target, computes each batch's laxity-derived priority, claims items in deadline × prefix order and submits them to vLLM as ordinary low-priority chat completions — while online clients (thick arrow) reach the engine directly, never through the gateway. Results settle idempotently, are metered at the batch discount, and are assembled into the output and error files the client downloads.

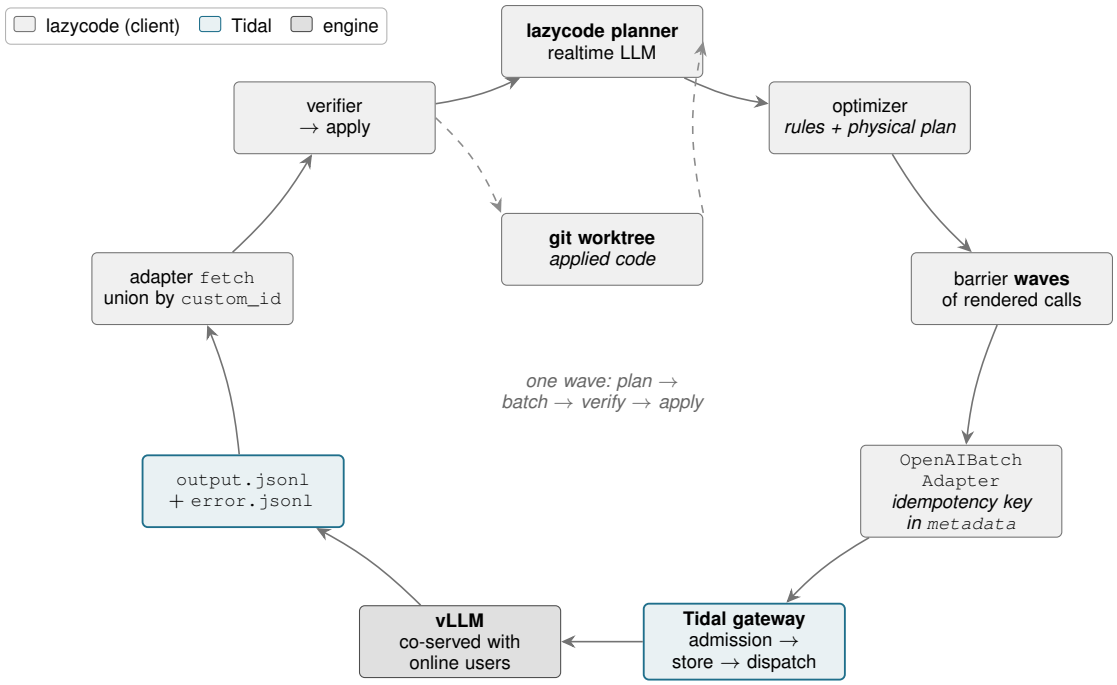

Fig. 11. **The complete loop.** A realtime model plans the work; the optimizer lowers the plan into barrier waves of rendered calls; the `OpenAIBatchAdapter` uploads a wave as JSONL and stamps a server-side idempotency key in the batch's `metadata`, so a crashed run can adopt its own orphaned batch. Tidal admits, stores and dispatches the wave into a vLLM that is simultaneously serving online users; the adapter fetches the output and error files and reassembles the wave by union over `custom_id`; the verifier runs the results and applies what passes into a git worktree, which feeds the next wave. This is the request lifecycle from client intent to applied code, entirely on self-hosted capacity at batch pricing. The client-side stages (planner, optimizer, waves, verifier) are detailed in the companion paper [28]; this figure shows only their interface to the serving tier.

TABLE VII
A-F FLEET VERSUS PINNED PLACEMENT, TWO PHASE-SHIFTED CPU REPLICAS.

arm	batch items in-window	batch tok/s	blended online p50 / p99
pinned (all batch → replica 0)	1884	24.4	1.44 s / 3.7 s
fleet (placement follows slack)	2494	32.2	3.41 s / 12.1 s

TABLE VIII
A-D SIGNAL MAPPING TO A DYNAMO-STYLE DISAGGREGATED DEPLOYMENT.

tidal signal	Dynamo counterpart
`kv_usage`	KVBM block metrics
`waiting`	router queue depth
observed rate	per-worker throughput

(median of 5). A duplicate arm thawing from the in-engine prefix cache measured $48.2\times$; we quote the cross-process number because durability — surviving process death — is the property a deadline-deferred prefill needs. Short prompts (208 tokens) get $1.9\times$, so the $\approx$1k-token crossover is an interpolation between two measured points. Load-bearing caveats: debug-grade connector; all-or-nothing exact-prompt-hash keying (partial prefix reuse is LMCache/Mooncake territory [18, 19]); GPU KV-connector mode forces piecewise cudagraphs.

c) A-D: signal mapping.: Table VIII. A-D needs no new policy, only new signals; the non-rename nuance is targeting prefill-worker troughs.

APPENDIX D
FULL DEADLINE-STRESS GRID

TABLE IX
ALL DEADLINE-STRESS RUNS (TECHNIQUE A, 15-MINUTE WINDOWS).

run	online load	items	completed	makespan	min. priority
saturated, best-effort	3.0 req/s	900	900/900	692 s	100
easy, best-effort	2.0 req/s	350	350/350	204 s	100
easy, laxity	2.0 req/s	350	350/350	188 s	91
tight, best-effort	2.6 req/s	500	500/500	351 s	100
tight, laxity (+floor)	2.6 req/s	500	500/500	328 s	66

ACKNOWLEDGMENT

Portions of this manuscript's text were drafted with the assistance of AI tools; the author reviewed all content and takes full responsibility for it.

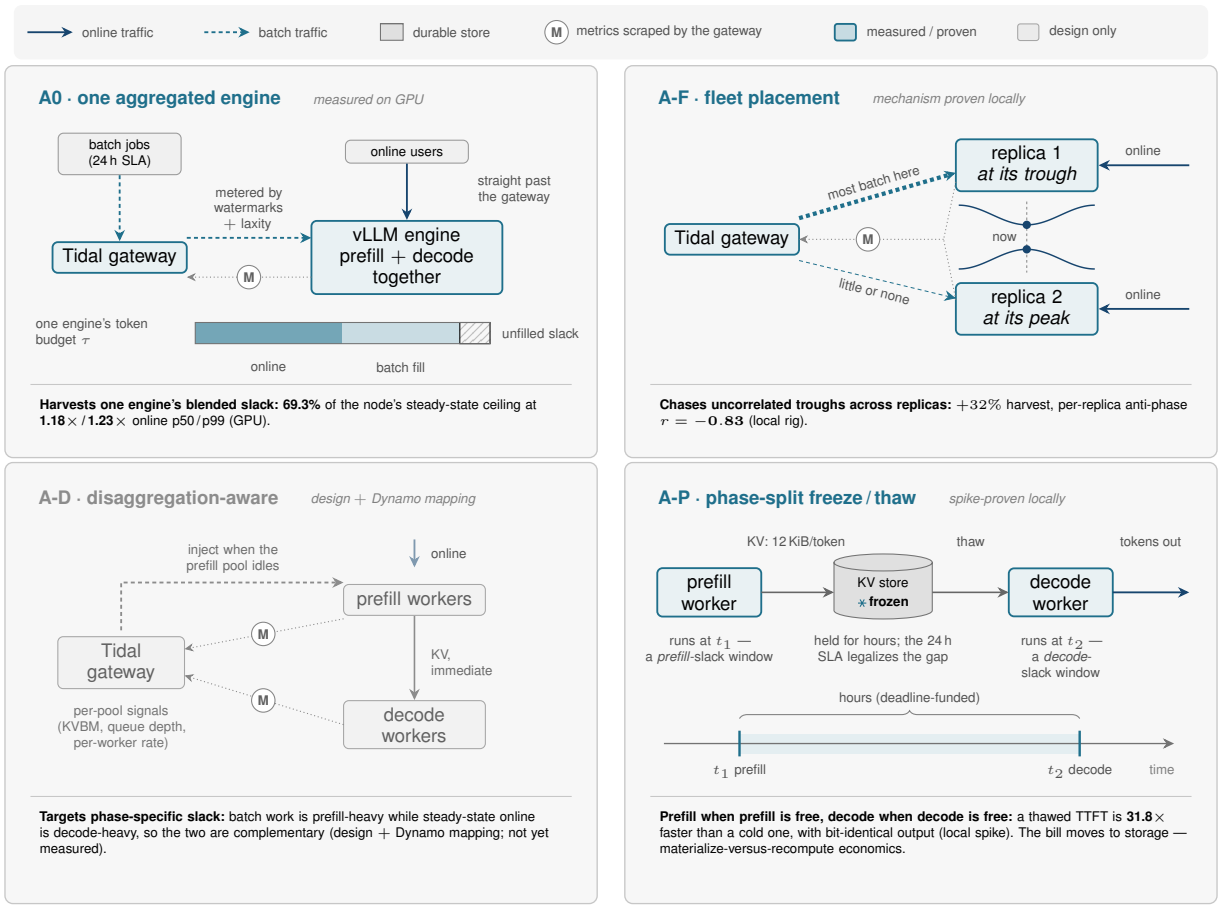

Fig. 12. **The A-ladder, drawn.** The four rungs of Table VI, in order of widening scope: A0 places deferrable work inside one aggregated engine, A-F across the replicas of a fleet, A-D into the idle *phase pool* of a disaggregated deployment, and A-P across *time* itself, by freezing a finished prefill's KV to a durable store and thawing it into a decode worker hours later. Each panel carries the advantage that rung buys and the number that evidences it; A-D is drawn dimmed because it is design plus a metric mapping (Table VIII) rather than a measurement. The last rung makes the section's one-liner literal: *disaggregation separates prefill from decode in space; a deadline-contracted batch tier separates them in time.*

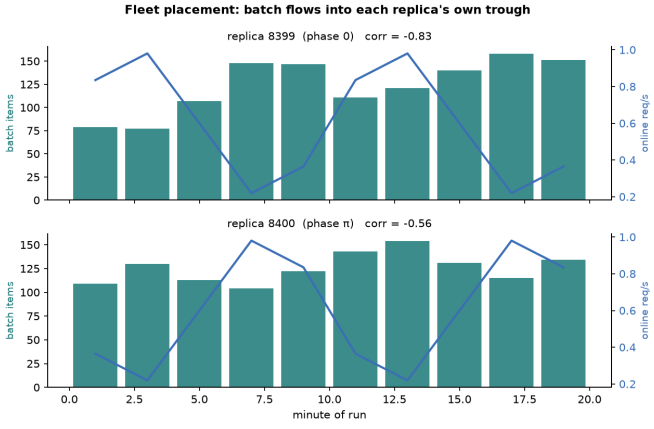

Fig. 13. A-F mechanism, per replica: measured batch completions (bars) against each replica's own analytic online rate (line). Volume splits evenly; *timing* follows each replica's trough.

References

[1] A. Agrawal et al. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. OSDI 2024. arXiv:2403.02310.

[2] S. Williams, A. Waterman, D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM* 52(4):65–76, 2009.

[3] G.-I. Yu et al. Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI 2022.

[4] B. Wu et al. Fast Distributed Inference Serving for Large Language Models. arXiv:2305.05920.

[5] B. Sun et al. Llumnix: Dynamic Scheduling for Large Language Model Serving. OSDI 2024. arXiv:2406.03243.

[6] Y. Zhong et al. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving. OSDI 2024. arXiv:2401.09670.

[7] P. Patel et al. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. ISCA 2024. arXiv:2311.18677.

[8] T. Sun, P. Wang, F. Lai. HyGen: Efficient LLM Serving via Elastic Online-Offline Request Co-location. arXiv:2501.14808.

[9] Y. Qiao et al. ConServe: Fine-Grained GPU Harvesting for LLM Online and Offline Co-Serving. arXiv:2410.01228.

[10] Y. Zhao et al. BlendServe: Optimizing Offline Inference for Auto-regressive Large Models with Resource-aware Batching. arXiv:2411.16102.

[11] Niyama: Breaking the Silos of LLM Inference Serving. arXiv:2503.22562.

[12] FREESH: Fair, Real-time, Energy-Efficient Scheduling for LLM Serving. arXiv:2511.00807.

[13] QLM: Queue Management for SLO-Oriented Large Language Model Serving. SoCC 2024. arXiv:2407.00047.

[14] Ascendra: Dynamic Request Prioritization for Efficient LLM Serving. arXiv:2504.20828.

[15] SCORPIO: Serving the Right Requests at the Right Time. arXiv:2505.23022.

[16] Online-Offline Co-location for LLM Serving. arXiv:2511.21862.

[17] Echo: Efficient Co-scheduling of Hybrid Online-Offline Tasks. arXiv:2504.03651.

[18] R. Qin et al. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. arXiv:2407.00079.

[19] LMCache: a KV-cache layer for LLM serving (prefix reuse, tiered storage, transfer engine). github.com/LMCache/LMCache.

[20] NVIDIA Dynamo: a datacenter-scale distributed inference serving framework, 2025. github.com/ai-dynamo/dynamo.

[21] A. K. Mok. Fundamental Design Problems of Distributed Systems for the Hard- Real-Time Environment. MIT, 1983.

[22] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180.

[23] R. Abhyankar et al. InferCept: Efficient Intercept Support for Augmented LLM Inference. ICML 2024. arXiv:2402.01869.

[24] vLLM RFC: Priority Scheduling. github.com/vllm-project/vllm/issues/6077.

[25] vLLM RFC: SLA-Tiered Scheduling (closed, not planned). github.com/vllm-project/vllm/issues/30256.

[26] vLLM PR: OpenAI-compatible online Batch and Files API (open). github.com/vllm-project/vllm/pull/44445.

[27] lazycode: plan with a realtime LLM, execute on batch APIs. github.com/rajagurunath/lazycode.

[28] G. Lunkupali Venugopal. LazyCode: Compiling Interactive Intent into Batch Execution — a Query-Optimizer Architecture, Its Economics, and the Limits of Overnight Coding Agents. Companion paper, 2026.